Assignment 1 - Git & GitHub Fundamentals
Report Template

- ***Full Name:*** Hanae Cherkaoui
- ***Student ID:*** 162125
- ***Course Code & Section:*** CSC3324 03
- ***GitHub Username:*** HanaeCherk90

1. Repository Setup

Briefly explain how you initialized the repository and what you ignored in .gitignore.

I created a folder named git-session1-HanaeCherk90, copied the starter files into it and ran `git init`. I created `.gitignore` before the first `git add` with three rules: `local.env` (local configuration), `*.log` (normal log files) and `!important.log` (keeps that file visible). `git check-ignore -v` confirmed that local.env and app.log are ignored and that important.log is not.

2. README & Commit History

Briefly describe your README and how you organized your commits.

The README has my name, GitHub username, a short description, a technologies list, one code block, one link and the provided banner image. I made small logical commits: starter files, .gitignore, README, styling, app.js, the About section, a merge commit, a README edit made on GitHub, and an extra style change. That is 9 commits across app.js, style.css, index.html, README.md, .gitignore and important.log.

3. Selective Staging

Explain how you changed two files but committed only one of them first.

I edited style.css and app.js at the same time, then ran `git add style.css` only. `git status` showed style.css under "Changes to be committed" and app.js under "Changes not staged for commit". I committed style.css first, left app.js modified, and committed it afterwards as a separate commit.

4. Branch & Merge

Briefly explain the feature branch you created and how you merged it into main.

I created `about-feature` with `git switch -c about-feature`, added an About section to index.html and committed it on that branch. I switched back to main and ran `git merge --no-ff about-feature`, which created a visible merge commit so the branch appears in the history graph.

5. Remote Changes

Explain the difference between commit and push, then fetch and pull.

Commit vs push: a commit records staged changes in my local repository history only. A push uploads those local commits to the remote repository on GitHub.

Fetch vs pull: fetch downloads the remote's new commits and updates origin/main without touching my working files. Pull is a fetch followed by a merge, so it also updates my local branch and files.

I edited README.md on GitHub, then ran `git fetch`: Git said main was behind origin/main by 1 commit and my local README was unchanged. `git pull origin main` then fast-forwarded and brought in the edit.

```
$ git fetch
   8e1c668..634bb43  main -> origin/main
$ git status
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
$ git pull origin main
Fast-forward
 README.md | 2 +-
```

6. Fair Use of AI

I used Claude (an AI assistant) as a step-by-step guide during the lab. I asked it to explain the order of the tasks, what commands such as `git switch -c`, `git merge --no-ff`, `git fetch` and `git diff --staged` do, and to help when something went wrong (my project folder was empty so `git add` failed, a multi-line README command did not finish in the terminal, and the README looked wrong on GitHub). I accepted suggestions that matched the assignment text and changed others, for example I kept my own commit messages and README wording. I verified every answer by running the commands and checking `git status`, `git log`, `git check-ignore` and the GitHub page.

7. Sources & Reflection

List any useful sources you used. Then write 2-3 sentences on the hardest concept and what you would check first if Git behaves unexpectedly.

The hardest concept was the difference between the working directory, the staging area and a commit, because a file can be modified but not staged, and a commit only contains what was staged. If Git behaves unexpectedly, I would first run `git status` to see my branch and what is staged, then `git log --oneline --graph --all` to see where my branch and the remote are.